

```

import pandas as pd
import json

accounts = {}
next_account_number = 1001

MIN_BALANCE = 100.00
DAILY_WITHDRAWAL_LIMIT = 500.00

def generate_account_number():
    global next_account_number
    acc_num = str(next_account_number)
    next_account_number += 1
    return acc_num

def record_transaction(account_number, type, amount, current_balance, target_account=None):
    transaction = {
        'type': type,
        'amount': amount,
        'timestamp': pd.Timestamp.now().strftime('%Y-%m-%d %H:%M:%S'),
        'current_balance': current_balance
    }
    if target_account:
        transaction['target_account'] = target_account
    accounts[account_number]['transactions'].append(transaction)

def create_account():
    global accounts
    account_number = generate_account_number()
    accounts[account_number] = {
        'balance': 0.0,
        'transactions': [],
        'daily_withdrawal_today': 0.0 # To track daily limit
    }
    print(f"Account {account_number} created successfully with a starting balance of $0.00")

def deposit(account_number, amount):
    if account_number not in accounts:
        print("Error: Account not found.")
        return False
    if amount <= 0:
        print("Error: Deposit amount must be positive.")
        return False

    accounts[account_number]['balance'] += amount
    record_transaction(account_number, 'Deposit', amount, accounts[account_number]['balance'], None)
    print(f"Deposit of ${amount:.2f} successful. New balance for {account_number}: ${accounts[account_number]['balance']:.2f}")
    return True

def withdraw(account_number, amount):
    if account_number not in accounts:
        print("Error: Account not found.")
        return False
    if amount <= 0:
        print("Error: Withdrawal amount must be positive.")
        return False
    if accounts[account_number]['balance'] - amount < MIN_BALANCE:
        print(f"Error: Insufficient funds. A minimum balance of ${MIN_BALANCE:.2f} must be maintained.")
        return False
    if accounts[account_number]['daily_withdrawal_today'] + amount > DAILY_WITHDRAWAL_LIMIT:
        print(f"Error: Daily withdrawal limit of ${DAILY_WITHDRAWAL_LIMIT:.2f} exceeded.")
        print(f"You have already withdrawn ${accounts[account_number]['daily_withdrawal_today']:.2f} today.")
        return False

```

```

accounts[account_number]['balance'] -= amount
accounts[account_number]['daily_withdrawal_today'] += amount
record_transaction(account_number, 'Withdrawal', amount, accounts[account_number]['balance'])
print(f"Withdrawal of ${amount:.2f} successful. New balance for {account_number}: ${accounts[account_number]['balance']:.2f}")
return True

def transfer(from_account, to_account, amount):
    if from_account not in accounts or to_account not in accounts:
        print("Error: One or both accounts not found.")
        return False
    if from_account == to_account:
        print("Error: Cannot transfer to the same account.")
        return False
    if amount <= 0:
        print("Error: Transfer amount must be positive.")
        return False
    if accounts[from_account]['balance'] - amount < MIN_BALANCE:
        print(f"Error: Insufficient funds in account {from_account}. A minimum balance of {MIN_BALANCE} is required.")
        return False

    # Perform withdrawal from source account
    accounts[from_account]['balance'] -= amount
    record_transaction(from_account, 'Transfer Out', amount, accounts[from_account]['balance'])

    # Perform deposit to target account
    accounts[to_account]['balance'] += amount
    record_transaction(to_account, 'Transfer In', amount, accounts[to_account]['balance'])

    print(f"Transfer of ${amount:.2f} from {from_account} to {to_account} successful.")
    print(f"New balance for {from_account}: ${accounts[from_account]['balance']:.2f}")
    print(f"New balance for {to_account}: ${accounts[to_account]['balance']:.2f}")
    return True

def get_balance(account_number):
    if account_number not in accounts:
        print("Error: Account not found.")
        return
    print(f"Account {account_number} balance: ${accounts[account_number]['balance']:.2f}")

def get_mini_statement(account_number):
    if account_number not in accounts:
        print("Error: Account not found.")
        return
    if not accounts[account_number]['transactions']:
        print(f"No transactions found for account {account_number}.")
        return

    print(f"\n--- Mini Statement for Account {account_number} ---")
    for t in accounts[account_number]['transactions']:
        target_info = f" to {t['target_account']}" if 'target_account' in t and t['type'] == 'Withdrawal' else f" from {t['target_account']}" if 'target_account' in t and t['type'] == 'Deposit' else ""
        print(f"{t['timestamp']} | Type: {t['type']}{target_info} | Amount: ${t['amount']:.2f}")
    print("-----")

def reset_daily_withdrawal_limits():
    # This function would ideally be called once per day by a cron job or similar.
    # For this simulation, we'll just provide an option to reset it manually.
    for acc_num in accounts:
        accounts[acc_num]['daily_withdrawal_today'] = 0.0
    print("Daily withdrawal limits reset for all accounts.")

def save_accounts_to_json(filename='bank_data.json'):
    try:
        with open(filename, 'w') as f:
            json.dump(accounts, f, indent=4)
        print(f"Account data saved to {filename}")
    except Exception as e:
        print(f"Error saving account data: {e}")

```

```

except Exception as e:
    print(f"Error saving accounts data: {e}")

def load_accounts_from_json(filename='bank_data.json'):
    global accounts, next_account_number
    try:
        with open(filename, 'r') as f:
            loaded_accounts = json.load(f)
            accounts.update(loaded_accounts)

        # Update next_account_number to avoid conflicts with loaded accounts
        if accounts:
            max_acc_num = max(int(acc_num) for acc_num in accounts.keys())
            next_account_number = max_acc_num + 1
        print(f"Account data loaded from {filename}")
    except FileNotFoundError:
        print(f"No existing account data found at {filename}. Starting with empty accounts")
    except json.JSONDecodeError:
        print(f"Error decoding JSON from {filename}. File might be corrupted.")
    except Exception as e:
        print(f"Error loading accounts data: {e}")

def delete_account(account_number):
    if account_number not in accounts:
        print("Error: Account not found.")
        return False
    if accounts[account_number]['balance'] != 0.0:
        print(f"Error: Account {account_number} has a non-zero balance of ${accounts[account_number]['balance']}")
        return False

    del accounts[account_number]
    print(f"Account {account_number} deleted successfully.")
    return True

def display_menu():
    print("\n--- Mini Banking System Menu ---")
    print("1. Create Account")
    print("2. Deposit")
    print("3. Withdraw")
    print("4. Transfer Funds")
    print("5. Check Balance")
    print("6. Get Mini Statement")
    print("7. (Admin) Reset Daily Withdrawal Limits")
    print("8. Load Account Data")
    print("9. Save Account Data")
    print("10. Delete Account")
    print("11. Exit")
    print("-----")

def main():
    load_accounts_from_json() # Load data at startup
    while True:
        display_menu()
        choice = input("Enter your choice: ").strip()

        if choice == '1':
            create_account()
        elif choice == '2':
            acc_num = input("Enter account number: ").strip()
            try:
                amount = float(input("Enter deposit amount: ").strip())
                deposit(acc_num, amount)
            except ValueError:
                print("Invalid amount. Please enter a number.")
        elif choice == '3':
            acc_num = input("Enter account number: ").strip()
            try:

```

```

        amount = float(input("Enter withdrawal amount: ").strip())
        withdraw(acc_num, amount)
    except ValueError:
        print("Invalid amount. Please enter a number.")
elif choice == '4':
    from_acc = input("Enter source account number: ").strip()
    to_acc = input("Enter target account number: ").strip()
    try:
        amount = float(input("Enter transfer amount: ").strip())
        transfer(from_acc, to_acc, amount)
    except ValueError:
        print("Invalid amount. Please enter a number.")
elif choice == '5':
    acc_num = input("Enter account number: ").strip()
    get_balance(acc_num)
elif choice == '6':
    acc_num = input("Enter account number: ").strip()
    get_mini_statement(acc_num)
elif choice == '7':
    reset_daily_withdrawal_limits()
elif choice == '8':
    load_accounts_from_json() # User can manually load again if needed
elif choice == '9':
    save_accounts_to_json()
elif choice == '10':
    acc_num = input("Enter account number to delete: ").strip()
    delete_account(acc_num)
elif choice == '11':
    print("Exiting Mini Banking System. Goodbye!")
    break
else:
    print("Invalid choice. Please try again.")

# Run the banking system
main()

```

Account data loaded from bank_data.json

```

--- Mini Banking System Menu ---
1. Create Account
2. Deposit
3. Withdraw
4. Transfer Funds
5. Check Balance
6. Get Mini Statement
7. (Admin) Reset Daily Withdrawal Limits
8. Load Account Data
9. Save Account Data
10. Delete Account
11. Exit
-----

```

Account 1002 created successfully with a starting balance of \$0.00.

```

--- Mini Banking System Menu ---
1. Create Account
2. Deposit
3. Withdraw
4. Transfer Funds
5. Check Balance
6. Get Mini Statement
7. (Admin) Reset Daily Withdrawal Limits
8. Load Account Data
9. Save Account Data
10. Delete Account
11. Exit
-----

```

Account 1003 created successfully with a starting balance of \$0.00.

```

--- Mini Banking System Menu ---

```

```
1. Create Account
2. Deposit
3. Withdraw
4. Transfer Funds
5. Check Balance
6. Get Mini Statement
7. (Admin) Reset Daily Withdrawal Limits
8. Load Account Data
9. Save Account Data
10. Delete Account
11. Exit
```

```
-----
Error: Account not found.
```

```
--- Mini Banking System Menu ---
```

```
1. Create Account
2. Deposit
3. Withdraw
4. Transfer Funds
5. Check Balance
6. Get Mini Statement
7. (Admin) Reset Daily Withdrawal Limits
8. Load Account Data
9. Save Account Data
10. Delete Account
```

✓ Mini Banking System

This system simulates basic banking operations including account creation, deposits, withdrawals, and transfers. It tracks transaction history, enforces business rules like minimum balance and daily withdrawal limits, and provides a menu-driven interface.

```
a = {
  rt': {
    'description': "Welcome to the Mysterious Adventure!\nYou find yourself at the entrance of a dark,
    'choices': {
      '1': {'text': "Take the narrow, overgrown trail.", 'score_effect': 10, 'next_scene': 'p_forest'},
      '2': {'text': "Take the wider, well-trodden road.", 'score_effect': 5, 'next_scene': 'lage_road'}
    },
    'prompt': "Which path do you choose?"
  },
  p_forest': {
    'description': "You venture deeper into the forest. You hear strange noises.",
    'choices': {
      '1': {'text': "Investigate the sound.", 'score_effect': 20, 'next_scene': 'ruin_explore'},
      '2': {'text': "Try to find a clearing.", 'score_effect': -5, 'next_scene': 'river_bank'}
    },
    'prompt': "What do you do?"
  },
  lage_road': {
    'description': "The wider road leads you to a small, deserted village.",
    'choices': {
      '1': {'text': "Search the houses for supplies.", 'score_effect': 15, 'next_scene': 'ruin_explore'},
      '2': {'text': "Continue past the village.", 'score_effect': -10, 'next_scene': 'final_scene'}
    },
    'prompt': "What's your next move?"
  },
  n_explore': {
    'description': "Inside the ruins, you find a glimmering artifact.",
    'choices': {
      '1': {'text': "Take the artifact!", 'score_effect': 30, 'ending': 'ancient_treasure_found'},
      '2': {'text': "Leave it be.", 'score_effect': 5, 'ending': 'peaceful_exit'}
    },
    'prompt': "Your decision?"
  },
  er_bank': {
```

```

'description': "You reach a wide river. There's a rickety bridge (1) or you could try to
'choices': {
    '1': {'text': "Cross the bridge.", 'score_effect': 10, 'ending': 'bridge_crossed'},
    '2': {'text': "Swim across.", 'score_effect': -15, 'ending': 'exhausted_crossing'}
},
'prompt': "How do you cross?"

al_choice_village': {
'description': "After exploring the village, you see a distant tower (1) and a winding ro
'choices': {
    '1': {'text': "Head towards the tower.", 'score_effect': 25, 'ending': 'tower_discover
    '2': {'text': "Follow the winding road.", 'score_effect': 5, 'ending': 'road_to_nowhe
},
'prompt': "Where do you go next?"

al_choice_road': {
'description': "After a long walk on the endless road, you find a hidden cave (1) and not
'choices': {
    '1': {'text': "Enter the cave.", 'score_effect': 15, 'ending': 'hermit_wisdom'},
    '2': {'text': "Follow the glimmer.", 'score_effect': -5, 'ending': 'mirage_trap'}
},
'prompt': "What catches your attention?"

data = {
    'discovered_treasure': {
        'text': "You discovered an ancient artifact and its power changed your destiny forever!",
        'title': "The Collector's Fate"

    'peaceful_exit': {
        'text': "You left the ancient ruins undisturbed and found inner peace. You eventually fo
        'title': "The Peaceful Wanderer"

    'river_crossed': {
        'text': "You successfully navigated the perils of the deep forest and crossed the river,
        'title': "The Survivor"

    'exhausted_crossing': {
        'text': "Despite losing some belongings, you made it across the river. You're safe, but t
        'title': "The Weary Traveler"

    'tower_discovery': {
        'text': "You reached the mysterious tower and uncovered its secrets, gaining immense know
        'title': "The Enlightened"

    'road_to_nowhere': {
        'text': "You followed the winding road, but it led to nowhere significant. Your journey c
        'title': "The Endless Journey"

    'hermit_wisdom': {
        'text': "The wise hermit shared ancient knowledge, guiding you to a path of enlightenmen
        'title': "The Seeker of Truth"

    'mirage_trap': {
        'text': "You were led astray by illusions and ended up back where you started, feeling a
        'title': "The Deceived"

def _scene(current_scene_key, score, decisions):
    scene_data = game_data[current_scene_key]
    print(f"\n{scene_data['description']}")

    # Display choices
    scene_options = list(scene_data['choices'].keys())
    for option in scene_options:

```

```

choice_prompts = []
for option in choice_options:
    choice_prompts.append(f"({option}) {scene['choices'][option]['text']}")
t(f"{scene['prompt']} {' / '.join(choice_prompts)}")

while True:
    player_choice = input("Your choice: ").strip()
    if player_choice in choice_options:
        break
    else:
        print(f"Invalid choice. Please choose from: {' / '.join(choice_options)}")

next_path = scene['choices'][player_choice]
score += chosen_path['score_effect']
decisions.append(f"{current_scene_key.replace('_', ' ').title()}: Chose {player_choice} ({chosen_path['score_effect']})")

if 'next_scene' in chosen_path:
    return play_scene(chosen_path['next_scene'], score, decisions)
elif 'ending' in chosen_path:
    return end_game(score, decisions, chosen_path['ending'])
else:
    print("Error: Scene configuration missing next_scene or ending.")
    return end_game(score, decisions, "unknown_error_ending")

def game(score, decisions, ending_type):
    t("\n--- Game Over ---")
    t(f"Your final score: {score}")
    t("Your journey's ending:")

    ending_info = endings_data.get(ending_type, {"text": "An unexpected ending.", "title": "Mysterious Ending"})
    t(ending_info["text"])
    t(f"Ending: {ending_info['title']}")

    t("\nYour decisions along the way:")
    for d in decisions:
        print(f"- {d}")

    _again = input("\nDo you want to play again? (yes/no): ").strip().lower()
    if _again == 'yes':
        main_game_loop()
    else:
        print("Thanks for playing!")

def _game_loop():
    _scene('start', 0, [])

if __name__ == '__main__':
    main_game_loop()

```

```

Welcome to the Mysterious Adventure!
You find yourself at the entrance of a dark forest.
Which path do you choose? (1) Take the narrow, overgrown trail. / (2) Take the wider, well-trodden path.
Your choice: 1

You venture deeper into the forest. You hear strange noises.
What do you do? (1) Investigate the sound. / (2) Try to find a clearing.
Your choice: 2

You reach a wide river. There's a rickety bridge (1) or you could try to swim across (2).
How do you cross? (1) Cross the bridge. / (2) Swim across.
Your choice: 2

--- Game Over ---
Your final score: -10
Your journey's ending:
Despite losing some belongings, you made it across the river. You're safe, but the journey was long and tiring.
Ending: The Weary Traveler

```

Your decisions along the way:

- Start: Chose 1 (Take the narrow, overgrown trail.)
- Deep Forest: Chose 2 (Try to find a clearing.)
- River Bank: Chose 2 (Swim across.)

Do you want to play again? (yes/no): NO

Thanks for playing!